

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Бэк-энд разработка

Отчет

Практическая работа №2

Выполнила:

Красноперова Елизавета

К3343

Проверил:

Добряков Д. И.

Санкт-Петербург

2026 г.

Задача

- Опишите все функциональные и нефункциональные требования к вашему бэкенд-приложению
- Выберите формат реализации API, которого вы будете придерживаться: REST API или Backend For Frontend
- Спроектируйте все эндпоинты вашего API в формате OpenAPI-схемы с учётом реализованной диаграммы БД
- Опишите тело каждого запроса и тело каждого ответа, продумайте возможные ошибки, возвращаемые из API

Ход работы

1 Анализ требований

На основе технического задания и диаграммы базы данных, разработанной в ДЗ1, был составлен список функциональных требований к бэкенд-приложению:

1. Аутентификация – регистрация пользователя, вход в систему с получением JWT.
2. Личный кабинет – просмотр и редактирование своих рецептов, просмотр сохраненных рецептов и подписок;
3. Поиск рецептов – текстовый поиск, фильтрация по типу блюда, сложности, времени готовки, поиск по ингредиентам;
4. Страница рецепта – информация с фото, видео, пошаговыми инструкциями, ингредиентами, лайками и комментариями;
5. Социальные функции – лайки, подписки/отписки, комментарии, сохранение рецептов в закладки.

Нефункциональные требования: формат JSON, аутентификация через JWT Bearer Token, пагинация списков, единый формат ошибок, проверка прав доступа (403 при попытке редактирования чужих ресурсов).

2 Выбор формата API

В качестве архитектурного стиля выбран REST API. Решение обосновано тем, что ресурсы сервиса (пользователи, рецепты, комментарии, подписки) естественно отображаются на REST-сущности, а стандартные HTTP-методы (GET, POST, PUT, DELETE) однозначно соответствуют CRUD-операциям.

3 Проектирование эндпоинтов

Были спроектированы следующие группы эндпоинтов:

Группа	Методы	Назначение
Auth	POST /auth/register, POST /auth/login	Регистрация и вход
Recipes	GET/POST /recipes, GET/PUT/DELETE /recipes/{id}	Создание, поиск, просмотр,

		редактирование, удаление рецептов
Profile	GET/PUT /users/me, GET /users/me/recipes, GET /users/me/save	Личный кабинет: профиль, публикации, сохраненные
Social	POST/DELETE /recipes/{id}/like, POST/DELETE /recipes/{id}/save, GET/POST /recipes/{id}/comments, PUT/DELETE /comments/{id}, POST/DELETE /users/{id}/subscribe, GET /users/me/subscriptions, GET /users/me/subscribers	Лайки, сохранения рецептов, комментарии, подписки

4 Создание OpenAPI-спецификации

Спецификация оформлена в формате OpenAPI 3.0.3 на языке YAML. Документ содержит:

- Метаданные (info, version, servers)
- Схему безопасности BearerAuth (JWT)
- Описание всех эндпоинтов сгруппированных по тегам: Auth, Recipes, Profile, Social
- Структуры запросов и ответов описаны inline (непосредственно в методах)

Спецификация проверена во встроенном валидаторе Swagger Editor. Критических ошибок нет, все методы отображаются корректно.

Вывод

В результате выполнения домашнего задания спроектирован REST API для сервиса обмена рецептами и кулинарных блогов. Выбранный подход обеспечивает понятную структуру эндпоинтов, независимость клиента от сервера и возможность тестирования через Swagger UI.

Полный код документа:

```
openapi: "3.0.3"
info:
  title: "Сервис обмена рецептами"
  version: "1.0.0"
servers:
  - url: "/api"
```

components:

securitySchemes:

BearerAuth:

type: http

scheme: bearer

bearerFormat: JWT

paths:

/auth/register:

post:

tags: [Auth]

summary: Регистрация

requestBody:

required: true

content:

application/json:

schema:

type: object

required: [username, email, password]

properties:

username: { type: string }

email: { type: string, format: email }

password: { type: string, minLength: 6 }

responses:

"201": { description: "Создан" }

"400": { description: "Ошибка валидации" }

"409": { description: "Занят email/username" }

/auth/login:

post:

tags: [Auth]

summary: Вход

requestBody:

required: true

content:

application/json:

schema:

```
  type: object
  required: [email, password]
  properties:
    email: { type: string }
    password: { type: string }
  responses:
    "200": { description: "JWT-токен и профиль" }
    "401": { description: "Неверные данные" }
```

/recipes:

get:

```
  tags: [Recipes]
  summary: "Поиск с фильтрацией по типу, сложности, ингредиентам"
  parameters:
    - { name: search, in: query, schema: { type: string } }
    - { name: dish_type, in: query, schema: { type: string } }
    - { name: difficulty, in: query, schema: { type: string, enum: [easy, medium, hard] } }
    - { name: min_time, in: query, schema: { type: integer } }
    - { name: max_time, in: query, schema: { type: integer } }
    - { name: ingredients, in: query, schema: { type: string }, description: "ID через запятую (1,2,3)" }
    - { name: sort_by, in: query, schema: { type: string, enum: [published_at, likes_count, cooking_time_min] } }
    - { name: page, in: query, schema: { type: integer, default: 1 } }
    - { name: limit, in: query, schema: { type: integer, default: 20 } }
  responses:
    "200": { description: "Список рецептов" }
```

post:

```
  tags: [Recipes]
  summary: "Создать рецепт"
  security:
    - BearerAuth: []
  requestBody:
    required: true
    content:
      application/json:
        schema:
          type: object
          required: [title, cooking_time_min, difficulty, dish_type, steps, ingredients]
```

```
properties:
  title: { type: string }
  description: { type: string }
  photo_main_url: { type: string }
  video_url: { type: string }
  cooking_time_min: { type: integer }
  difficulty: { type: string, enum: [easy, medium, hard] }
  dish_type: { type: string }
steps:
  type: array
  items:
    type: object
    properties:
      step_number: { type: integer }
      instruction_text: { type: string }
      photo_url: { type: string }
ingredients:
  type: array
  items:
    type: object
    properties:
      ingredient_id: { type: integer }
      quantity: { type: string }
responses:
  "201": { description: "Рецепт создан" }
  "401": { description: "Нет токена" }
```

/recipes/{recipe_id}:

get:

tags: [Recipes]

summary: "Страница рецепта (шаги, ингредиенты, видео)"

parameters:

- { name: recipe_id, in: path, required: true, schema: { type: integer } }

responses:

"200": { description: "Рецепт" }

"404": { description: "Не найден" }

put:

```
tags: [Recipes]
summary: "Редактировать рецепт"
security:
  - BearerAuth: []
parameters:
  - { name: recipe_id, in: path, required: true, schema: { type: integer } }
requestBody:
  required: true
  content:
    application/json:
      schema:
        type: object
        properties:
          title: { type: string }
          description: { type: string }
          difficulty: { type: string }
responses:
  "200": { description: "Обновлён" }
  "401": { description: "Нет токена" }
  "403": { description: "Чужой рецепт" }
  "404": { description: "Не найден" }
```

```
delete:
tags: [Recipes]
summary: "Удалить рецепт"
security:
  - BearerAuth: []
parameters:
  - { name: recipe_id, in: path, required: true, schema: { type: integer } }
responses:
  "200": { description: "Удалён" }
  "401": { description: "Нет токена" }
  "403": { description: "Чужой рецепт" }
  "404": { description: "Не найден" }
```

```
/users/me:
get:
tags: [Profile]
```

summary: "Мой профиль"

security:

- BearerAuth: []

responses:

"200": { description: "Профиль" }

put:

tags: [Profile]

summary: "Редактировать профиль"

security:

- BearerAuth: []

requestBody:

content:

application/json:

schema:

type: object

properties:

username: { type: string }

email: { type: string }

bio: { type: string }

avatar_url: { type: string }

responses:

"200": { description: "Обновлён" }

/users/me/recipes:

get:

tags: [Profile]

summary: "Мои рецепты (публикации)"

security:

- BearerAuth: []

responses:

"200": { description: "Список рецептов" }

/users/me/saved:

get:

tags: [Profile]

summary: "Сохранённые рецепты"

security:

- BearerAuth: []

responses:

"200": { description: "Список рецептов" }

/recipes/{recipe_id}/like:

post:

tags: [Social]

summary: "Лайкнуть рецепт"

security:

- BearerAuth: []

parameters:

- { name: recipe_id, in: path, required: true, schema: { type: integer } }

responses:

"200": { description: "Лайк поставлен" }

"409": { description: "Уже лайкнут" }

delete:

tags: [Social]

summary: "Убрать лайк"

security:

- BearerAuth: []

parameters:

- { name: recipe_id, in: path, required: true, schema: { type: integer } }

responses:

"200": { description: "Лайк убран" }

/recipes/{recipe_id}/save:

post:

tags: [Social]

summary: "Сохранить рецепт в закладки"

security:

- BearerAuth: []

parameters:

- { name: recipe_id, in: path, required: true, schema: { type: integer } }

responses:

"200": { description: "Сохранён" }

"409": { description: "Уже сохранён" }

delete:

tags: [Social]

summary: "Убрать из сохранённых"

security:

- BearerAuth: []

parameters:

- { name: recipe_id, in: path, required: true, schema: { type: integer } }

responses:

"200": { description: "Удалён из сохранённых" }

/recipes/{recipe_id}/comments:

get:

tags: [Social]

summary: "Комментарии к рецепту"

parameters:

- { name: recipe_id, in: path, required: true, schema: { type: integer } }

responses:

"200": { description: "Список комментариев" }

post:

tags: [Social]

summary: "Оставить комментарий"

security:

- BearerAuth: []

parameters:

- { name: recipe_id, in: path, required: true, schema: { type: integer } }

requestBody:

required: true

content:

application/json:

schema:

type: object

required: [content]

properties:

content: { type: string }

parent_comment_id: { type: integer, nullable: true }

responses:

"201": { description: "Комментарий создан" }

/comments/{comment_id}:

put:

tags: [Social]

summary: "Редактировать комментарий"

security:

- BearerAuth: []

parameters:

- { name: comment_id, in: path, required: true, schema: { type: integer } }

requestBody:

required: true

content:

application/json:

schema:

type: object

properties:

content: { type: string }

responses:

"200": { description: "Обновлён" }

"403": { description: "Чужой комментарий" }

delete:

tags: [Social]

summary: "Удалить комментарий"

security:

- BearerAuth: []

parameters:

- { name: comment_id, in: path, required: true, schema: { type: integer } }

responses:

"200": { description: "Удалён" }

"403": { description: "Чужой комментарий" }

/users/{user_id}/subscribe:

post:

tags: [Social]

summary: "Подписаться на автора"

security:

- BearerAuth: []

parameters:

- { name: user_id, in: path, required: true, schema: { type: integer } }

responses:

"200": { description: "Подписка оформлена" }

"409": { description: "Уже подписаны" }

delete:

tags: [Social]

summary: "Отписаться от автора"

security:

- BearerAuth: []

parameters:

- { name: user_id, in: path, required: true, schema: { type: integer } }

responses:

"200": { description: "Отписка выполнена" }

/users/me/subscriptions:

get:

tags: [Social]

summary: "Мои подписки"

security:

- BearerAuth: []

responses:

"200": { description: "Список авторов" }

/users/me/subscribers:

get:

tags: [Social]

summary: "Мои подписчики"

security:

- BearerAuth: []

responses:

"200": { description: "Список подписчиков" }